

# CAC Controller

## Technical Documentation — Pioneer CAC CD Autochanger Web Controller

### Contents

- 1. Hardware and Connection
- 2. Pioneer CAC Serial Protocol
- 3. Software Architecture
- 4. Frontend
- 5. Data Flow
- 6. Database Details
- 7. JSON Import
- 8. Deployment
- 9. Security
- 10. Known Limitations

## 1. Hardware and Connection

### 1.1 Supported Models

| Model     | Capacity | Players | Baud Rate | Volume | Speed |
|-----------|----------|---------|-----------|--------|-------|
| CAC-V3000 | 300 CDs  | 2       | 9600      | Yes    | Yes   |
| CAC-V3200 | 300 CDs  | 2       | 9600      | Yes    | Yes   |
| CAC-V5000 | 500 CDs  | 2       | 9600      | Yes    | Yes   |
| CAC-V180M | 18 CDs   | 1       | 4800      | No     | No    |

### 1.2 Serial Connection

| Parameter | Value |
|-----------|-------|
|-----------|-------|

|                    |                                          |
|--------------------|------------------------------------------|
| Baud Rate          | 9600 (V3000/V3200/V5000) or 4800 (V180M) |
| Data Bits          | 8                                        |
| Stop Bits          | 1                                        |
| Parity             | None                                     |
| Terminator         | CR ( \r , ASCII 0x0D)                    |
| Max Command Length | 20 characters                            |

Physical connection: 15-pin D-Sub connector (RS-232C) on the changer, connected via a USB-to-Serial adapter to the Raspberry Pi.

## 1.3 System Architecture

```

Browser (Smartphone / Tablet / PC)
  |
  | HTTP (REST API) + WebSocket
  v
Node.js Server (Raspberry Pi)
  |
  | RS-232C Serial (9600 Baud, 8N1)
  v
Pioneer CAC CD Autochanger

```

## 2. Pioneer CAC Serial Protocol

### 2.1 Command Format

All commands are sent in the format `<Player-ID>PS<Command>\r`.

Example: `1PS001ZSPL\r` — selects player 1, loads disc 001, and starts playback.

### 2.2 Execution Commands (27)

| Command       | Code | Description   | Arguments |
|---------------|------|---------------|-----------|
| Player Select | PS   | Select player | 1 or 2    |
| Disc Select   | ZS   | Load CD       | 001-500   |
| Disc Return   | ZR   | Return CD     | —         |

|               |    |                |              |
|---------------|----|----------------|--------------|
| Start         | SA | Start motor    | —            |
| Reject        | RJ | Stop motor     | —            |
| Play          | PL | Playback       | —            |
| Pause         | PA | Pause          | —            |
| Scan Forward  | NF | Scan forward   | —            |
| Scan Reverse  | NR | Scan reverse   | —            |
| Search        | SE | Execute search | —            |
| Track         | TR | Select track   | 01-99        |
| Volume        | VL | Volume         | 0-255        |
| Speed         | SP | Speed          | 90-110%      |
| Fade          | FD | Volume fade    | 0-255 target |
| Duration      | DU | Fade duration  | 1-99 seconds |
| Stop Marker   | SM | Stop point     | —            |
| Changer Reset | !! | Reset changer  | —            |

## 2.3 Query Commands (14)

| Command      | Code | Response       | Description       |
|--------------|------|----------------|-------------------|
| Job Status   | ?J   | R / B          | Ready / Busy      |
| Player Mode  | ?P   | P01 - P22      | Current mode      |
| Disc Number  | ?Z   | 3-digit / XXX  | Loaded CD         |
| Track Number | ?R   | 2-digit / XX   | Current track     |
| Time Code    | ?T   | 4-digit (MMSS) | Elapsed time      |
| TOC Info     | ?Q   | 10-digit       | Tracks + lead-out |
| Model Name   | ?X   | 6 chars        | Model name        |
| Mech Error   | ?E   | E00 - E99      | Error codes       |
| Disc Status  | ?K   | 8 flags        | Disc status       |

## 2.4 Player Modes

| Code | Mode | Description          |
|------|------|----------------------|
| P01  | Park | CD loaded, motor off |

|     |            |                      |
|-----|------------|----------------------|
| P02 | Set Up     | TOC being read       |
| P03 | Reject     | Motor stopping       |
| P04 | Play       | Playback in progress |
| P06 | Pause      | Paused               |
| P07 | Search     | Searching            |
| P08 | Scan       | Scanning             |
| P20 | Disc Unset | No CD loaded         |
| P21 | Load       | CD being loaded      |
| P22 | Unload     | CD being returned    |

## 2.5 Volume Mapping

| dB     | Value | dB     | Value |
|--------|-------|--------|-------|
| 0 dB   | 255   | -30 dB | 138   |
| -1 dB  | 247   | -40 dB | 97    |
| -3 dB  | 239   | -50 dB | 60    |
| -6 dB  | 230   | -60 dB | 16    |
| -10 dB | 213   | -70 dB | 5     |
| -20 dB | 174   | Mute   | 0     |

## 3. Software Architecture

---

### 3.1 Technology Stack

| Component     | Technology       | Version |
|---------------|------------------|---------|
| Runtime       | Node.js          | ≥ 18.x  |
| Web Framework | Express          | 5.x     |
| WebSocket     | ws               | 8.x     |
| Database      | better-sqlite3   | 11.x    |
| Serial Comm.  | serialport       | 12.x    |
| Frontend      | Vanilla JS (SPA) | —       |

## 3.2 Project Structure

```
cac-controller/  
├─ server.js           # Entry point  
├─ package.json  
├─ cac-controller.service # Systemd unit  
├─ src/  
│   ├─ protocol.js     # Pioneer protocol  
│   ├─ serial.js        # Serial connection  
│   ├─ player.js        # Player manager  
│   ├─ scanner.js       # CD scanner  
│   ├─ musicbrainz.js   # MusicBrainz API  
│   ├─ database.js      # SQLite database  
│   ├─ routes.js        # REST API  
│   └─ websocket.js     # WebSocket manager  
├─ public/  
│   ├─ index.html       # SPA  
│   ├─ css/app.css      # Styling  
│   ├─ js/app.js        # Frontend logic  
│   ├─ js/i18n.js       # Translations  
│   └─ covers/          # Cover images  
└─ data/  
    └─ cac-controller.db # Database
```

## 3.3 Module Details

### server.js — Entry Point

Initializes all components: Database → Serial Connection → PlayerManager → Scanner → Express → WebSocket. Graceful shutdown via SIGINT/SIGTERM.

### protocol.js — Pioneer Protocol

Contains the complete protocol definition: model properties, command constants, response parser. The parser automatically recognizes error codes, modes, track/disc numbers, time codes, TOC info, and more.

### serial.js — Serial Connection

Manages the RS-232C connection with automatic reconnection (every 5 seconds), separate command queues per player (200ms delay), and priority queue for time-critical commands. Fire-and-forget architecture: commands and responses are completely decoupled.

## player.js — Player Manager

Central module (~800 lines). Manages both player states, polls the changer at staggered intervals, and implements four playback modes:

- Automatic switch to the other player on disc end
- Pre-cueing of the other player for seamless transitions
- Current CD, both players, or entire changer
- Server-side engine with intelligent dual-player management and CD pre-loading

## scanner.js — CD Scanner

Scans slots: Load CD → Wait → Start motor → Query TOC → Save data → Return CD. Real-time progress via EventEmitter/WebSocket. Pauses polling during scan.

## musicbrainz.js — MusicBrainz Integration

Rate-limited API calls (1.1s interval). Search by free text, artist+title, barcode, track count+duration, or disc ID. Cover art via Cover Art Archive. Automatic mapping of MusicBrainz data to internal format.

## database.js — SQLite Database

WAL mode for performance. UPSERT with COALESCE (only provided fields are updated). Cascading deletes for tracks. Default settings on first start.

## routes.js — REST API

50+ endpoints for player control, library CRUD, cover upload, MusicBrainz search, scanner, playlists, favorites, ratings, history, search, statistics, settings, and JSON import.

## websocket.js — WebSocket Manager

Broadcasts all state changes to all connected clients. Sends initial state on connection.

# 4. Frontend

---

## 4.1 Single Page Application

The frontend is a SPA without build tools or frameworks. Four files: `index.html` (structure), `app.js` (logic), `i18n.js` (translations), `app.css` (dark theme).

## 4.2 Internationalization

Three levels: HTML attributes ( `data-i18n` ), JavaScript ( `t('key')` ), server messages (structured objects). ~200 translation keys for German and English.

## 4.3 Real-time Updates

WebSocket delivers instant updates for player status, scanner progress, and playlist changes. Time display is interpolated client-side for smooth rendering.

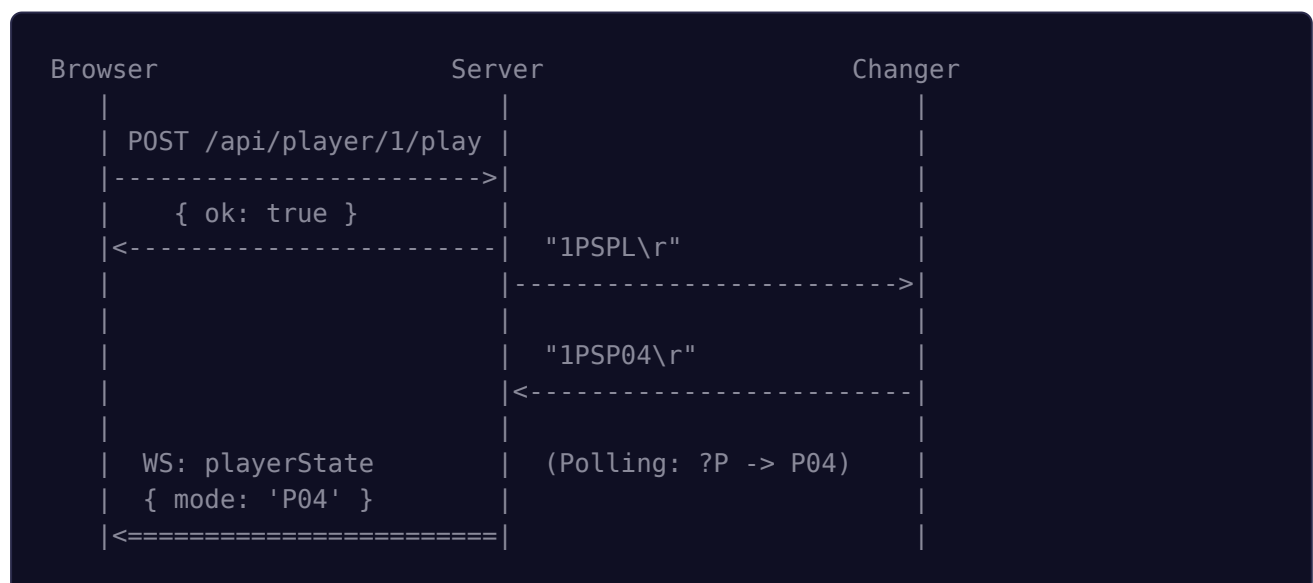
## 4.4 Cover Upload

Client-side processing: File selection/Drag&Drop → Canvas resize (max 500x500) → JPEG 85% → Base64 upload → Server saves to `public/covers/`.

# 5. Data Flow

---

## 5.1 Player Control



## 5.2 CD Scanning

| Browser                | Server                  | Changer |
|------------------------|-------------------------|---------|
| POST /api/scanner/scan |                         |         |
| { slot: 42 }           |                         |         |
| ----->                 | "1PS042ZS\r"            |         |
|                        | ----->                  |         |
| WS: scanProgress       |                         |         |
| { status: 'loading' }  | (waiting for P01/P02..) |         |
| <=====                 |                         |         |
|                        | "1PSSA\r"               |         |
| WS: scanProgress       | ----->                  |         |
| { status: 'reading' }  |                         |         |
| <=====                 | "1PS?Q\r"               |         |
|                        | ----->                  |         |
|                        | "1PS0112045000\r"       |         |
|                        | <-----                  |         |
| WS: scanProgress       | (TOC: 12 tracks, 45:00) |         |
| { totalTracks: 12 }    |                         |         |
| <=====                 | "1PSRJ\r" + "1PSZR\r"   |         |
|                        | ----->                  |         |

## 5.3 Playlist with Dual Player

Playlist: [Slot 42/Track 3, Slot 101/Track 1, Slot 42/Track 7]

Step 1: Load CD 42 into Player 1, play Track 3  
 Simultaneously: Load CD 101 into Player 2 (pre-load)

Step 2: CD 101 already in Player 2  
 Stop Player 1, play Track 1 on Player 2  
 Player 1 still has CD 42 (no pre-load needed)

Step 3: CD 42 still in Player 1  
 Stop Player 2, play Track 7 on Player 1

## 6. Database Details

### 6.1 Schema

```

cds (
  slot INTEGER PRIMARY KEY,      -- 1-500
  disc_id TEXT,                  -- MusicBrainz Disc ID
  title TEXT, artist TEXT,      -- Album metadata
  year TEXT, genre TEXT,
  total_tracks INTEGER,
  total_duration_seconds INTEGER,

```



```

cover_url TEXT,           -- Path to cover
musicbrainz_release_id TEXT,
barcode TEXT, label TEXT, country TEXT,
created_at TEXT, updated_at TEXT
)

tracks (
  slot INTEGER REFERENCES cds(slot) ON DELETE CASCADE,
  track_number INTEGER,
  title TEXT, artist TEXT,
  duration_seconds INTEGER, isrc TEXT,
  UNIQUE(slot, track_number)
)

playlists, playlist_items, play_history,
favorites, ratings, settings

```

## 6.2 WAL Mode

Write-Ahead Logging for better performance with concurrent reads and writes.

## 6.3 UPSERT with COALESCE

Only provided fields are updated — existing values are preserved when NULL.

## 7. JSON Import

---

### 7.1 Supported Formats

Array of CDs, object with `cds` array, or object with slot numbers as keys. Automatic field mapping for various naming conventions ( `slot / cd_number / nr` , `title / album / name` , etc.).

### 7.2 Preview UI

Editable table with status badges (New/Update/Warning), editable slot numbers (auto-swap on duplicates), per-CD checkboxes, and mojibake repair.

## 8. Deployment

---

## 8.1 Systemd Service

```
[Service]
Type=simple
User=pi
WorkingDirectory=/opt/cac-controller
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
SupplementaryGroups=dialout
Environment=NODE_ENV=production
```

## 8.2 Installation

```
git clone https://github.com/NFDiJee/cac-controller.git /opt/cac-controller
cd /opt/cac-controller
npm install
sudo cp cac-controller.service /etc/systemd/system/
sudo systemctl enable --now cac-controller
```

## 9. Security

---

- No authentication (designed for LAN use only)
- No HTTPS (unencrypted communication)
- Recommendation: Firewall, no port forwarding, optionally Nginx with Basic Auth

## 10. Known Limitations

---

1. The Pioneer only provides total duration and track count. Individual track lengths only via MusicBrainz.
  2. Loading a CD takes 10-30 seconds. Full scan  $\approx$  several hours.
  3. Current version controls one changer. Multi-changer (hub architecture) is planned.
  4. Audio output is analog on the device. No digital streaming.
-

CAC Controller v1.0.0 — Copyright © 2025 Dirk Jensen

Based on the Pioneer CAC-V3000 Programming Manual (Version 2.0, May 1993)

MIT License